

ZeeBeam: The Zero-Knowledge Beam

A zkVM Proof of a Beacon-Seeded Capture-Log Relation with Learned Scores and a Zcash Receipt

Authors. Cathal Ryan Hynes (PolieBotics). Drafted with BOSUN, the project's automated research assistant (a Claude-family model running the project's desk), whose role is stated in the Acknowledgements.

Status. Public release manuscript after nineteen referee passes by a second model (rounds 1 to 9, a tenth verification pass on the additions, an eleventh publication-byte pass at ultra effort, a twelfth verification pass on the applied text and seven closing passes, every finding adjudicated); final relation proved for all 259 anchored rows; session chain proved. Every measurement is taken from the project records cited in Appendix F. Every citation was checked against arXiv, the IACR ePrint archive, DBLP or the publisher on 2 September 2026. ZeeBeam names the capture-and-proof system; the work belongs to Dark Lantern, the wider privacy and zero-knowledge research programme (Section 1). The subtitle states the claim.

Abstract

We describe a projector-camera capture protocol in which each projected pattern is a deterministic integer function of a hash chain that folds in a public randomness beacon (drand quicknet) and the camera's previous frame, and we present one zkVM program (SP1, wrapped in Groth16) for the following relation over the operator's recorded bytes, two proofs of concrete session rows under the relation's previous revision, and the reproducible build and circuit-independent execution check of its final revision: for one row of a 712-row session, the beacon signatures of that row and of the preceding row verify under the quicknet key; the chain state was advanced from the preceding row's record exactly as the protocol specifies; the pattern supplied to the coupling computation is the one derived from that state; two frozen integer neural networks, a coupling discriminator between the raw 24.5 MB sensor frame and that pattern and an eleven-class pose classifier on the frame, produced the published outputs; the row is a leaf of the committed session tree; a supplied Zcash v6

transaction, recomputed from raw bytes with its branch checked to a supplied block header (canonical-mainnet status is an out-of-band check), carries an encrypted memo whose `binding=` value, parsed after in-circuit decryption under the operator's viewing key, equals SHA-256 of a receipt that opens a chameleon commitment over a chain-log prefix containing the row. The final statement is 1,101 bytes; the raw Groth16 proof is 356 bytes and verifies with an open crate against a 32-byte verification-key hash. What the proof establishes is execution binding: these outputs were computed by exactly these pinned functions on exactly these bytes, whose hashes sit in a chain seeded by publicly verifiable randomness. What it does not establish, and we say so: that the bytes came from a camera, that the learned scores mean what their names suggest, or that the chameleon anchor binds a unique record against its own holder. We give the security statements under explicit assumptions, report the learned components as diagnostics with their confounds, including a time-aliasing effect that a predeclared temporal-separation test narrowed but did not remove, and report two proofs of concrete session rows at 12 min 50 s and 14 min 11 s on one A100. Those two proofs are of the previous revision of the relation, which verified the row's own beacon round only (1,085-byte statement); the final revision, which also verifies the preceding row's round and publishes both, was then built reproducibly on a second machine, which reproduced the ELF and key byte for byte, and proved for the same two rows (14 min 17 s and 12 min 30 s on one A100); every statement byte of those two proofs equals the statement the program produced when executed against circuit-independent oracles. A second proof covers the whole session chain: one beacon signature for each of its 66 rounds, all 712 transitions and the session tree, in one eight-minute proof. All 259 rows the August anchor covers were executed under the final program and matched their oracles on every byte, and all 259 were then Groth16-proved under the same key: 257 proofs made on eight A100 instances plus the two of ceremony 2, every proof verified standalone and every proved statement byte-identical to its executed one.

1. Introduction

Generative models have made pixels cheap. Pixels alone no longer provide reliable evidence of physical capture, and the existing answers each cover part of the gap. Hardware attestation (a signing key in the camera) proves which device signed, not what it saw. Content credentials such as C2PA [C2PA] bind edits to an origin assertion but rest on the origin's honesty. Timestamping [Haber and Stornetta 1991; OpenTimestamps] proves that a digest existed by a block and says nothing about the physical world at that moment. Zero-knowledge proofs of image transformations [Naveh and Tromer 2016; Kang et al. 2022b; Datta et al. 2024] prove that a published image derives from a signed original and inherit the original's attestation. Active-

illumination liveness checks [Gerstner and Farid 2022] make a scene answer a challenge but produce no transferable proof.

The idea of the ZeeBeam protocol is to make the illumination the challenge and the recording the response, and then to prove the whole response computation. A projector shows a pattern derived from a hash chain; the chain folds in a public randomness beacon and the hash of the camera's previous frame; the camera records; a frozen discriminator scores the coupling between frame and pattern; a frozen classifier reads a pose; and one zkVM program re-derives everything from the recorded bytes and commits a public statement.

Names. ZeeBeam names the capture-and-proof system: the protocol of Section 4, the projector-camera rig, the relation and the proving pipeline. A ZeeBeam recording is a recorded session with rows proved under that system; the one reported here comprises rows 1 to 259 of a 712-row session (Section 7). Dark Lantern names only the wider privacy and zero-knowledge research programme this work belongs to, and names no component of the artifact. `TB-v0.9`, the `ZB...` magic strings and the `zeebeam_*` crate and path names are frozen identifiers of the proved bytes.

What this paper claims. The contribution is the proof, and precisely this much:

1. **Execution binding of a multi-leg relation (Theorem 1).** One SP1 program verifies two beacon signatures, re-derives the chain state and the pattern, hashes the raw frame, runs both frozen integer networks, checks session membership, parses a Zcash transaction from raw bytes to block header, decrypts its memo, and checks a chameleon opening; every cross-component link is an equality on values the program itself computed. A verifier needs the 356-byte proof, the 1,101-byte statement and a 32-byte key hash. Both revisions of the relation have been proved for the same two rows; the final revision's key was reproduced byte for byte on the proving machine (Section 7.1, 7.3).
2. **Correct temporal indexing of the beacon bound (Proposition 2, conditional on an application premise).** The pattern against which the proved frame is checked derives from the state the *preceding* row produced, so the beacon round that bounds it is the preceding row's. An earlier version of this relation verified only the row's own round, which bounds the *following* frame; a referee pass caught it (Section 7.4) and the relation now verifies both rounds and publishes both. The physical conclusion, that the frame was recorded after that round, additionally needs premise A8 and the acquisition and model assumptions of Section 3 (P1, P2), which the proof cannot supply.
3. **In-circuit decryption of a shielded memo as a binding primitive (Proposition 3).** The receipt that pins the commitment is identified only inside an encrypted Orchard/Ironwood memo. We decrypt it in circuit under the

operator's incoming viewing key and argue that alternate keys cannot yield a different binding without breaking the note commitment's binding property.

4. **An inclusion receipt with disclosed equivocation (Proposition 4).** The Zcash anchor is a chameleon commitment whose trapdoor the operator holds. We present it as an inclusion receipt with a disclosed contingency, not as an adversarial upper bound, and prove trapdoor knowledge so the contingency is exhibited rather than hidden.
5. **Measured cost, audits, controls and artifact.** 4.15 billion RISC-V instructions, 52% of them the projector render; two source audits and nineteen manuscript referee passes by a second model, most of which found real defects; thirty-odd negative controls; a bundle a stranger can verify with the bundled verifier crate and its direct `sp1-verifier` and `sha2` dependencies.
6. **The session chain proved (Section 9.5).** A second program verifies one beacon signature for each of the 66 rounds, checks every row's beacon value against it, recomputes all 712 transitions of the chain log and rebuilds the session tree, in 369 million instructions and one eight-minute proof; the per-row proofs and the chain proof share the context digest and root.
7. **Every row through the oracles; every anchored row through the relation (Section 7.6).** All 712 rows through the circuit-independent oracles; all 259 anchored rows executed under the final ELF and matched on every statement byte; all 259 then Groth16-proved under the final key (257 on an eight-instance fleet, Section 7.1), each proof standalone-verified and each proved statement identical to the executed one (`all_rows/proofs/`, `PINS_all_rows.json`).
8. **The learned components reported as diagnostics.** Section 8 gives training configurations, exact splits, seeds and ranges, the time-aliasing confound, a predeclared temporal-separation test that narrowed but did not remove it, and the realness measurements with their small samples and an outstanding adaptive attack.

2. Background

zkVMs and SP1. A zkVM proves correct execution of an ordinary program compiled to a RISC-V target. We use SP1 v6.4.0 [Succinct Labs 2026]: a STARK core prover, a recursion layer, and a Groth16 wrapper [Groth 2016] whose verifying key for circuit version v6.1.0 is embedded in the `sp1-verifier` crate. A program's identity is its verification-key hash, a function of the compiled ELF.

drand quicknet. The League of Entropy's quicknet beacon [drand] publishes every 3 s a BLS12-381 signature [Boneh, Lynn and Shacham 2001] over the round number under a fixed group key (scheme `bls-unchained-g1-rfc9380`: 48-byte G1 signatures, 96-

byte G2 key, hash-to-curve per RFC 9380 [Faz-Hernández et al. 2023] with DST BLS_SIG_BLS12381G1_XMD:SHA-256_SSWU_RO_NUL_; genesis 1692803367). The randomness is SHA-256 of the signature. Its public key and chain hash are compiled into the guest and were checked against the live API on 2 September 2026.

BLAKE3. A tree hash with an extendable-output mode [Aumasson et al. 2020]; the protocol uses it with domain separation for the chain advance, the session tree and the tile tree, and its XOF to expand a per-channel seed into a pattern stream.

Zcash Orchard, Ironwood, ZIP-244. Orchard is Zcash's Pallas-curve shielded pool [ZIP 224]; Ironwood is the Version 6 bundle of NU6.3 [ZIP 229]. Transaction identifiers are BLAKE2b-256 digest trees over the transaction's bundles [ZIP 244; ZIP 229 for v6]. Notes are encrypted to a diversified address with ChaCha20-Poly1305 under a BLAKE2b-derived key from a Pallas key agreement; decryption with an incoming viewing key recomputes the note commitment and compares it to the on-chain value [Zcash protocol specification §4.20].

Chameleon commitments. $C = m \cdot G + r \cdot Y$ over a prime-order group with $Y = td \cdot G$ can be opened to any message by whoever knows td [Krawczyk and Rabin 2000]. Profile R of the ZeeBeam design uses this deliberately as repudiable custody [project note]. The group is the prime-order subgroup of Baby Jubjub [Whitehat, Bellés and Baylina 2020] over the BN254 scalar field.

3. Definitions, system model and threat model

3.1 Objects and indices

A **session** is one run of the rig producing rows $0..N-1$ (here $N = 712$). At **row** t the projector shows the **emission** $E_t = \text{render}(S_t)$, the camera records **frame** raw_t (the 24,472,000-byte Bayer plane exposed while E_t is shown), and the operator samples a beacon round r_t with randomness v_t . The **chain advance** produces S_{t+1} from $(S_t, \text{BLAKE3}(\text{raw}_t), \text{meta}_t, r_t, v_t)$. Hence E_t depends on r_{t-1} (through S_t), and r_t first affects E_{t+1} . The **chain log** records for each row: S_t , $\text{BLAKE3}(\text{raw}_t)$, $\text{BLAKE3}(E_t)$, meta_t , r_t , v_t and the beacon signature σ_t . The **response** of row t is the pair (raw_t, E_t) as scored by the frozen coupling discriminator.

3.2 Parties

The **operator** runs the rig and later acts as prover. The **verifier** receives a proof and statement, has the pinned constants of Appendix A, and can read the public drand schedule and the Zcash chain.

3.3 Adversary

The operator is the adversary. We assume the operator **can**: choose when to start and stop; present any scene; run any software on the rig, including injecting synthetic Bayer bytes in place of sensor output, substituting the projector, relaying another optical scene, or compromising the camera driver; choose which rows to prove; hold the chameleon trapdoor and the Orchard viewing key; and generate the session tree. We assume the operator **cannot**: forge quicknet signatures or obtain a round's signature before its scheduled time (threshold BLS, honest-majority group); rewrite mined Zcash blocks or cause a reorganisation deeper than the anchor's depth; find collisions in BLAKE3, BLAKE2b or SHA-256 or break the binding of Orchard's Sinsemilla-based note commitment; solve discrete logarithms in the Baby Jubjub subgroup; or break the knowledge soundness of SP1's STARK, recursion and Groth16 wrapper.

Explicitly out of scope (each is a real attack on any *physical* conclusion and none is addressed by the proof): sensor or driver compromise and digital raw injection; projector substitution and optical relay; adaptive attacks on the frozen models and poisoning of their training data; beacon threshold corruption or early share leakage; network-delay manipulation of beacon sampling; Zcash consensus attacks beyond ordinary reorganisation depth; verification-key substitution and compromised verifier distribution; key compromise; side channels. Section 9 states, for each theorem, which of these it does and does not survive.

3.4 What is and is not claimed

The proof establishes **execution binding** (Section 9.1): the published outputs are exactly what the pinned functions produce on the bytes whose hashes sit in the committed chain. Any **physical** reading, that a camera recorded a scene lit by an unpredictable pattern after a stated time, additionally requires A1 to A4, the application premise A8, P1 (genuine sensor acquisition) and P2's thresholded illumination inference (Section 9.2); none of these is established by the proof, and the reading is stated only in that conditional form. The learned components are ad hoc; the pose verdict in particular is a bound output of a fixed function and no more (Section 8).

4. The capture protocol (TB-v0.9)

Rig. A projector (1,920 × 1,080, RGB8) illuminates the scene; a 24.5-megapixel GenICam industrial camera (5,320 × 4,600 BayerRG8) driven through Aravis records it. The session ZEEBEAM_MAINNET_BLOCKING_TRAINING_300S_20260822_001 ran 301 s on 22

August 2026 (03:09:45 to 03:14:46 UTC) and produced 712 rows [session manifest]. The beacon refreshed 66 times, so a round covers about eleven rows on average.

Chain. With `S_0` a session seed and `TB:ROW:v9` the domain tag,

```
S_{t+1} = BLAKE3( "TB:ROW:v9" || S_t || u32be(32) || BLAKE3(raw_t) || u32be(28) || meta_t
                || u32be(8) || r_t (u64 BE) || u32be(32) || v_t )
seed_c   = BLAKE3(SEED_DOMAIN_c || S_t)                                c in {R, G, B}
stream_c = BLAKE3_X0F(seed_c, 43,110 bytes)
E_t      = four-octave fixed-point bilinear render of stream_c (1,920 x 1,080 x 3, uint8)
```

Every step is BLAKE3 or fixed-point integer arithmetic. `meta_t` is a 28-byte capture record (row index, frame id, timestamps).

Session commitment. A BLAKE3 Merkle tree of depth 10 over 712 leaves, `leaf_t = H(ROW, [context, t, S_t, BLAKE3(raw_t), meta_t, r_t, v_t, S_{t+1}, BLAKE3(E_t)])` with domain-separated padding leaves; the root is wrapped with the session context (identifier, `S_0`, `S_N`, authority-manifest digest, chain-log digest). The tree is built by the operator; the proof verifies one leaf's path and one transition (Section 5), not all 712 transitions (Section 9.5).

Anchor transactions. During and after the session, Zcash mainnet transactions were minted whose Ironwood memos carry a binding digest. The one used here, txid `8d1672...f206` in block 3456294, pins a receipt for a Profile-R chameleon commitment `C` over a `PREFIX.json` record describing the first 260 rows (their BLAKE3). According to the project records, the transaction was mined minutes after the 260 rows named by the supplied `PREFIX.json`; Proposition 4 does not turn that chronology into a bound. The receipt's network field reads `zcash-testnet`: a label inherited from the Profile-R testnet demonstrator's tooling, committed into the statement scalar `m` and therefore unchangeable after the fact; the anchor transaction itself is on mainnet (Section 7.4).

5. The statement and the relation

5.1 Public statement (1,101 bytes)

offset	field	encoding	constrained by the relation?
0..8	magic ZBROWM02	ASCII	fixed
8..12	ABI 3, protocol 9, terminal 1, reserved 0	u8 x4	fixed

offset	field	encoding	constrained by the relation?
12..16	public_bytes_declared = 876, the base program's own statement length (the appended blocks do not rewrite it)	u32 LE	fixed
16..20	row index t	u32 LE	equals the header's row and the leaf's index
20..28	row count 712, tree depth 10, session-id length 51	u32, u16, u16 LE	equal the witness fields the context digest is computed from
28..156	session identifier (51 bytes at 28..79, then zero padding)	ASCII, zero-padded	input to the context digest
156..188, 188..220	S_0, S_N	32 B each	inputs to the context digest
220..252, 252..284	authority-manifest SHA-256, chain-log BLAKE3	32 B each	inputs to the context digest (metadata; not recomputed in circuit)
284..316	context digest	32 B	recomputed in circuit from the fields above
316..348	ordered-session root	32 B	recomputed from the leaf and path
348..352	reserved	u8 ×4, zero	fixed
352..360	coupling header	u8 ×8	fixed
360..368	coupling score numerator	i64 LE	output of the frozen r32 network
368..376	denominator 4	u64 LE	fixed
376..408, 408..440	typed context, typed tile root	32 B each	recomputed from the preprocessed primaries
440..696	eight model and provenance digests	32 B ×8	fixed constants, checked against the embedded blob
696..704	magic ZBJP0SE1	ASCII	fixed
704..708, 708..712	pose verdict, head saturation	u32 LE	outputs of the frozen uncr64 network
712..800	eleven logit sums	i64 LE ×11	outputs
800..832	uncropped commitment	32 B	recomputed from the reduced frame

offset	field	encoding	constrained by the relation?
832..876	beacon magic ZBJBEAC1, verified = 1, quicknet chain hash	ASCII, u32 LE, 32 B	fixed; the flag is set only after verification
876..884	magic ZBZCINC3	ASCII	fixed
884..916, 916..948	anchor txid, block header hash	32 B each, internal byte order	recomputed from the raw transaction and header
948..956	magic ZBAUGST4	ASCII	fixed
956..988	contentRoot	32 B	recomputed from PREFIX.json
988..1020	receipt digest	32 B	recomputed; equals the decrypted memo binding
1020..1052, 1052..1084	C.x, C.y	32 B LE each	parsed from the pinned receipt, validated on-curve and in the subgroup
1084..1092	r_{t-1}	u64 BE	the round whose signature was verified for row $t-1$
1092..1100	r_t	u64 BE	the round whose signature was verified for row t
1100	trapdoor flag	u8	1 iff the private trapdoor was supplied and generates Y

The row's own S_t , S_{t+1} , raw and emission digests and v_t are computed and compared in circuit but not published; the statement carries the session root instead. Fields marked "fixed" are enforced by the guest under the pinned key; `PINS_final.json` records their expected bytes for an independent comparison against a decoded statement.

5.2 What each leg enforces

All arithmetic is integer; the guest contains no floating point. $H(D, \text{parts})$ denotes BLAKE3 over D followed by each part with a big-endian u32 length prefix.

Table 1. ZK all the things. Every check the guest enforces in circuit, one leg per row.

leg	relation
beacon (row t)	σ_t is a non-identity G1 point; $e(\sigma_t, G2) = e(\text{hash_to_G1}(\text{SHA-256}(r_t), \text{DST}), \text{pk_quicknet})$; $\text{SHA-256}(\sigma_t) = v_t$, the value the chain advance consumes; r_t published
chain advance	$\text{BLAKE3}(\text{raw}_t)$ computed over the 24,472,000-byte frame; $S_{\{t+1\}}$ as in Section 4
previous-row advance	row $t-1$'s record read from the anchored prefix; $\sigma_{\{t-1\}}$ verified likewise; $\text{SHA-256}(\sigma_{\{t-1\}}) = v_{\{t-1\}}$; $\text{advance}(S_{\{t-1\}}, \text{BLAKE3}(\text{raw}_{\{t-1\}}), \text{meta}_{\{t-1\}}, r_{\{t-1\}}, v_{\{t-1\}}) = S_t$; $r_{\{t-1\}}$ published; row 0 refused
emission	seed_c , XOF and render from S_t ; $\text{BLAKE3}(E_t)$
preprocess and typed root	RGGB pack once; PREPROCESS_V1 block means of a 1,024-square camera window (x_0 782, y_0 340 on the packed planes) and a 1,024-square emission window (x_0 448, y_0 28) to $4 \times 256 \times 256$ and $3 \times 256 \times 256$; 32-leaf depth-5 BLAKE3 tile tree giving (context, root)
coupling	frozen trained-r32 integer discriminator on the 8×8 -reduced pair; numerator over 4; embedded blob SHA-256 equals the compiled constant
pose	whole-plane half-up mean to $4 \times 256 \times 256$; commitment $\text{SHA-256}(\text{domain}, \text{spec}, t, \text{tag}, \text{bytes})$; 4×4 reduce; frozen uncr64 network; eleven int64 sums, first-maximum verdict, head saturation count
membership	leaf_t as in Section 4 from the journal's own values; ten siblings; wrapped root equals the committed root
Zcash inclusion	version 0x80000006, group 0xd884b698, branch 0x37a5165b; no transparent, Sapling or Orchard-V5 bundle; Ironwood action digests (compact, memo, non-compact); bundle digest excludes the anchor field; five-digest ZIP-244 txid with bare personalisations for absent bundles; 33-byte Merkle levels, direction in $\{0,1\}$, to header[36..68]; header hash SHA-256d
memo	orchard 0.15.3 <code>try_note_decryption</code> on action 0 under the 64-byte incoming viewing key; V3 plaintext; recomputed cmx equals the action's; derived esk regenerates epk; $\text{binding} = 64$ hex digits after <code>binding=</code>
prefix join	$\text{BLAKE3}(\text{260-row prefix})$ equals the digest in PREFIX.json; the proved row's S_t , Bayer digest, r_t , v_t inside the prefix equal the journal's; $\text{contentRoot} = \text{SHA-256}(\text{domain}, \text{u64be}(\text{len}), \text{PREFIX.json})$
receipt	$\text{SHA-256}(\text{receipt})$ equals the decrypted binding; Y, C parsed from canonical decimals (no leading zero, ≤ 77 digits, below the modulus), on-curve, $L \cdot P = O$; <code>recordScope</code> is sha256: + 64 lowercase hex
chameleon	$m = \text{SHA-512}(\text{domain}, 0x00, \text{canonical JSON } \{\text{contentRoot}, \text{network}, \text{profile}, \text{recordScope}\}) \bmod L$; $C = m \cdot \text{Base8} + r \cdot Y$, Base8 compiled in, r canonical below L

leg	relation
trapdoor	td canonical below L; flag = (td $\neq$ 0); if set, Y = td·Base8

5.3 Why the bindings hold (informal)

Beacon to chain: $v_t = \text{SHA-256}(\sigma_t)$ with σ_t verified for r_t , so $S_{\{t+1\}}$ depends on a verified beacon; and by the previous-row leg, S_t itself is recomputed from a verified $\sigma_{\{t-1\}}$, so E_t depends on a verified beacon. Chain to frame: $\text{BLAKE3}(\text{raw}_t)$ enters $S_{\{t+1\}}$ and the leaf, and the same packed planes feed both networks. Row to session: every leaf field comes from the journal the guest produced. Row to anchor: the proved row's fields are compared inside the anchored prefix bytes; contentRoot is computed from those bytes; the receipt is pinned by the memo binding; the memo by the action's cmx and the txid ; the txid by the header the verifier looks up. Commitment: Base8 is constant and points are validated in the prime-order subgroup. The formal statements are in Section 9.

6. Implementation

The guest is Rust for SP1's RISC-V target. BLS12-381 uses the `sp1-patches` fork of `bls12_381` (tag `patch-0.8.0-sp1-6.2.0`); there is no pairing syscall in SP1 6.4.0, so the pairing is software over accelerated field arithmetic (4.9 million instructions per verification). Orchard decryption uses `orchard` 0.15.3 and `zcash_note_encryption` 0.4.2 unmodified with default features off; the transitive `jubjub` 0.10.0 crate is vendored with four `const` field initialisers made non-`const` because the `bls12_381` fork's scalar constructors are not `const` fn (`RedJubjub` never executes). Baby Jubjub arithmetic uses `ark-bn254` 0.4 scalar-field types. The build passes `--remap-path-prefix` for the cargo home and the tree root and `--locked`, so the ELF embeds no machine path (Section 7.3).

Cost (row 96 under the final ELF: 4,149,712,293 instructions, 269,904 syscalls, 61.5 s host execution; row 72 executes 4,148,971,330 under the same ELF in 58.6 s (`final_relation/execution_logs/`); the region names are the guest's own cycle-tracker labels; `preprocess_and_pack` includes the nested `rggb_pack`):

region	instructions	share
emission render (four-octave integer)	2,141,120,868	51.6%
raw-frame BLAKE3 (24.5 MB)	702,715,828	16.9%
preprocess and pack (of which RGGB pack 229,464,788)	421,045,347	10.1%
pose network (uncr64 integer)	344,916,129	8.3%

region	instructions	share
coupling score (r32 integer)	338,997,588	8.2%
August anchor legs (prefix, receipt, chameleon, trapdoor, point validation)	126,039,564	3.0%
memo decryption (orchard, software Pallas)	15,394,564	0.4%
typed tile root	14,810,611	0.4%
previous-row advance (BLS verify + BLAKE3 advance)	6,733,568	0.2%
drand BLS verify (row t)	4,904,590	0.1%
XOF expansion	4,829,169	0.1%
txid + Merkle + header	235,697	<0.1%
membership	121,601	<0.1%
other: untracked glue inside the row relation 27,764,103 (header parse, chain advance, comparisons), public commitment 65,149, input reads 4,137, outside any region 13,780	27,847,169	0.7%
total	4,149,712,293	100%

Removing the renderer's cost would roughly halve the *instruction count*; whether a cheaper deterministic generator preserves the coupling discriminator's separation is an empirical question we have not tested.

Oracle discipline. The host compares the committed statement against expectations built without the circuit: session digests from the records; chain fields and both rounds from the chain log; pose verdict and logit sums from the audited parity vectors; typed context and root, uncropped commitment, coupling numerator and head saturation from Python (torch) re-runs of the audited modules and of both frozen networks reconstructed from their blobs (every layer's integrity hash re-validated; all 144 audited parity numerators and row 96's frozen outputs reproduced first); anchor identifiers from the block and transaction records; C from the receipt decoded in Python. For both proved rows all statement bytes are checked; none rests on the circuit alone. Two development incidents shaped this: the raw frame and its RRGB-packed planes have identical byte length, so an early version scored the wrong image and was caught only by the oracle; and an oracle constant was once a 70-digit truncation of a 76-digit decimal, where the circuit was right. The rule since is that an oracle mismatch is resolved from source, never by copying the circuit's output.

7. Evaluation

7.1 Proving

One Lambda `gpu_1x_a100_sxm4` (A100-SXM4-40GB, 30 vCPU, 216 GiB), SP1 6.4.0 CUDA prover (`sp1-gpu-server`), whole pipeline in one pass, twice on 2 September 2026.

Ceremony 1 (05:30 to 06:17 UTC) proved the relation as it then stood (1,085-byte statement, without the previous-row leg, before the path-independent build).

Ceremony 2 (15:59 to 16:36 UTC, a second instance of the same type) rebuilt the frozen final tree with `--locked`, reproduced the final ELF and key byte for byte (Section 7.3), and proved the final relation for the same two rows. In ceremony 2 the first proving call failed within eight seconds, before the GPU server's socket was up, and was retried once; the retry is the proof reported. Both ceremony-2 proving processes then panicked in the `sp1-cuda` client's destructor ("there is no reactor running") after their proofs and manifests were written and verified in-process, and were terminated by signal 6; the `stderr` logs are archived, and the proofs verify independently (Section 7.2).

ceremony	row	setup	prove (core + recursion + Groth16)	verify + 3 tamper controls	host RSS	artifacts
1 (1,085 B, vkey 0x0092cba2...)	72	21.4 s	850.97 s (14 min 11 s)	0.37 s	111.5 MiB (114,160 KiB)	framed 2,780 B; raw proof 356 B
1	96	21.4 s	769.58 s (12 min 50 s)	0.51 s	111.6 MiB (114,260 KiB)	framed 2,779 B; raw proof 356 B
2 (1,101 B, vkey 0x0019d16c...)	96	21.2 s	857.42 s (14 min 17 s)	0.37 s	111.1 MiB (113,760 KiB)	framed 2,795 B; raw proof 356 B
2	72	22.0 s	749.81 s (12 min 30 s)	0.36 s	111.6 MiB (114,240 KiB)	framed 2,795 B; raw proof 356 B

GPU memory in use, as read by the operator with `nvidia-smi` during proving (the prover logs no memory figure; `final_relation/GPU_MEMORY_OBSERVATIONS.md`), was about 26.5 GB in ceremony 1 and 25,615 to 27,279 MiB in ceremony 2. The ceremony-1 box ran from about 05:30 UTC to the termination request at 06:17 UTC, about 47 minutes at \$1.99 per hour; the ceremony-2 box ran 37 minutes including bootstrap, transfer and a 2 min 10 s CUDA host build, about \$1.23. The proved ceremony-2 statements are byte-identical to the statements the final ELF produced when executed on the

development machine (Section 7.5), so the proofs attest exactly the executed statements.

All anchored rows (2 to 3 September 2026). Eight further instances of the same type were launched at about 19:50 UTC on 2 September and driven from the development machine from 20:43 UTC. Each received the frozen tree, the 257 row witnesses and the anchor records, read the raw frames from the attached copy of the session, built the CUDA host from the frozen tree (whose build script rebuilds the guest), and printed the embedded ELF digest and vkey, which matched the pins on all eight boxes before any proof was made (ELF_REPRODUCED_ON_BOX in the orchestrator logs). One box first built and proved the chain relation (Section 9.5). The 257 anchored rows not proved in ceremony 2 were split into eight shards of 32 rows (33 on the chain box). At 23:10 UTC a network drop on the development machine killed the proving processes, which were children of SSH sessions, after 99 proofs had been written and verified in-process; the orchestrators recorded their boxes as finished without terminating them, and their pulls were incomplete. At 23:58 UTC the proving was restarted detached from any session, skipping rows whose proof existed and had verified; the first restarted proof began at 00:10 UTC. No row needed its single permitted retry. Over the 257 proofs the prove step took 727.8 s to 955.7 s (12 min 8 s to 15 min 56 s; median 755.6 s), setup 22.7 s (median), in-process verification with three tamper controls 0.36 s (median), host RSS 110.9 to 333.5 MiB, 54.6 GPU-hours of proving in total. Every process exited with the destructor panic described above after writing and verifying its proof. The boxes finished between 03:51 and 04:31 UTC on 3 September and were terminated between 03:55 and 04:33 UTC once their proofs had been pulled and the trapdoor and viewing-key files on them shredded: 4,073 instance-minutes, about \$135 at \$1.99 per hour, of which roughly 13.5 instance-hours were idle: the interval before dispatch and the incident-to-restart gap. On the development machine every one of the 257 proofs was then verified with the standalone verifier under vkey 0x0019d16c...0490, its 1,101-byte statement compared byte for byte with the executed statement of Section 7.6 (257 of 257 equal), and rows 1, 95, 130, 200 and 259 were verified cold with the SDK against the reproducible ELF and

their own oracles (all_rows/proofs/STANDALONE_VERIFY.txt, all_rows/logs/, PINS_all_rows.json, all_rows/proofs/FLEET_STATS.json).

fleet, 3 Sep 2026	rows	setup (median)	prove, min / median / max	verify + 3 tamper controls (median)	host RSS	artifacts
8 × A100, final key 0x0019d16c...	257	22.7 s	727.8 s / 755.6 s / 955.7 s	0.36 s	110.9 to 333.5 MiB	framed 2,791, 2,792, 2,793, 2,794, 2,795 B; raw proof 356 B each

7.2 Verification

Sizes, together. Raw Groth16 proof 356 B including its 4-byte vkey-hash prefix; framed SP1 proof object 2,779 B (row 96) and 2,780 B (row 72), bincode; statement 1,085 B (ceremony 1) and 1,101 B (ceremony 2); framed ceremony-2 proofs 2,795 B; verification-key hash 32 B.

Each proof verifies with the bundled standalone verifier (cryptographic dependency sp1-verifier 6.4.0; it also uses sha2 0.10 to print digests) from three inputs: the 356-byte raw proof (a 4-byte Groth16 vkey-hash prefix plus the encoded proof), the statement (1,085 bytes for ceremony 1; 1,101 for ceremony 2), and the SP1 vkey hash (0x0092cba2c77fe7fc426bb539add5d0365e4edea340c2f3c040da9bd73cf9a82c for ceremony 1; 0x0019d16cfe911d943315c04235c792fcccc3e0759aa3095a941b8508178e0490 for ceremony 2). The standalone verifier rejects a flipped statement byte and a wrong key. Both proofs were also cold-verified on a second machine with the SP1 SDK against the exact ELF that was proved.

7.3 Reproducibility of the verification key

Two builds of the same sources, lockfile and rustc 1.94.0-dev on two machines produced ELF's differing by 152 bytes, exactly 32 embedded cargo registry paths (/home/ubuntu against /home/c), and therefore different verification keys. The two proofs are pinned to the ELF that was proved (sha256 5b3ebfcd...4ce5). The final relation is built with both prefixes remapped and --locked; it embeds no machine path, and its ELF (1,171,416 bytes, sha256

8bcadf5373742e92eb36535680e4b9f092c00fdcf9a8f7171678c49b170f11db) has vkey 0x0019d16cfe911d943315c04235c792fcccc3e0759aa3095a941b8508178e0490. It produces the same statement bytes as the proved ELF on both rows apart from the added fields; whether it reproduces byte for byte on the second machine was done in ceremony 2, which reproduced the ELF and key byte for byte before proving (Section 7.1).

Line numbers are in the key. After the final tree was frozen, a five-line replacement of a three-line header comment in `memo.rs` (no code change) rebuilt to a different ELF of the same size (`sha256 485fd45e...7781`, `vkey 0x008f8897...b243`); reverting the comment restored `8bcadf53...11db` and `0x0019d16c...0490`. Panic-location records (`core::panic::Location`) compile the line number of every expect, unwrap and indexing site into the binary, so any edit that shifts lines changes the key, comments included. The frozen tree is therefore kept byte for byte as proved (its digest list is in the bundle), and that comment, which states the memo leg's key commitment without the A5 qualifier of Section 9.3, is corrected only in the post-freeze tree. **Second machine.** The ceremony-2 box (Section 7.1) built the frozen tree with `--locked` and reproduced the ELF and key byte for byte before proving [ceremony 2 log].

7.4 Audits and negative controls

A second model (OpenAI GPT-5.6 via Codex, reasoning effort high for rounds 1 to 10 and ultra thereafter; referred to in the project as "Sol") audited the staged sources twice and has refereed this manuscript nineteen times: nine rounds that closed before v3.0, a tenth, verification-only pass on the additions of Sections 7.6 and 9.5, an eleventh, ultra-effort pass on the publication bytes (v3.4), and a twelfth, verification-only pass on the applied text that adjudicated every earlier finding, and seven closing passes. The first source audit passed the fold and asked for parser hardening and an archived fixture tool. The second returned BLOCK on three findings, all confirmed: `Base8` was a private witness (with `C` and `m` fixed, opening 0, trapdoor 0 and $B8 = m^{-1} \cdot C$ pass with identical public output); `commitment_x` alone did not identify an Edwards point; the memo digest was a host input with no proved relation to the transaction. The first manuscript round found within its first hour that the verified round bounded the following frame rather than the proved one (Section 1, item 2); the referee reports are published with this release (`referee_trail/`, Appendix F) and this version answers them point by point. Model review is not independent validation; it is a second reader that has been useful.

One defect of the frozen host is disclosed rather than fixed, because fixing it would change the ELF (Section 7.3): when an independent coupling expectation is supplied, the host compares the numerator against it and counts the bytes as checked (as `oracle_bytes_circuit_derived = 0` records), but its hard-coded note still says the numerator "is circuit-derived". The ceremony-2 row-72 manifest and the row-95 fixture carry a correction sidecar saying so.

Every control below was run against the same binary as its positive and rejected:

leg	controls
beacon	round + 1 with the same signature
previous-row advance	row $t-1$'s signature altered inside the anchored prefix (rejects in that leg, before the prefix digest is checked)
Zcash inclusion	truncated transaction; non-minimal CompactSize; direction byte 2; tampered sibling; byte flipped inside the memo section; wrong expected txid
prefix join	one hex digit of row 96's S_t changed with the prefix BLAKE3 recomputed so only the row comparison can object; a byte flipped elsewhere; wrong contentRoot; wrong round
receipt and chameleon	a byte flipped anywhere in the receipt; a digit changed inside the public-key coordinate; wrong memo digest; wrong content scalar; wrong opening; opening 0
trapdoor	wrong trapdoor; wrong opening; wrong content scalar; wrong public key
memo	viewing-key byte flipped; tampered ciphertext (fails at the Merkle root, since the ciphertext is inside the txid)
cross-leg	an unrelated PREFIX.json field altered so the prefix digest still passes but contentRoot changes: the chameleon leg rejects
unit tests	leading-zero decimal; decimal equal to the modulus; 78-digit decimal; scalar equal to L; the order-2 point (0, -1); recordScope with escapes or upper-case hex

One control initially passed when it should have failed: the host sanitised the direction byte before the guest saw it. The host now passes raw bytes through.

7.5 Two rows

row	pose cue (annotation)	model verdict	agrees	beacon rounds published (r_{t-1} , r_t)	role in the pose model's data	role in the coupling model's data
96	02_superman (hold_confirmed, 4.9 s after cue)	2 (letter_y)	no	31,521,620 / 31,521,620	training (calibration) row	architecture-selection row
72	01_neutral (hold_confirmed, 14.9 s after cue)	0 (neutral)	yes	31,521,616 / 31,521,616	evaluation row	training (fit) row

Cue delays (seconds_since_cue 4.932 s and 14.932 s) are from the cue-aware compliance annotation, which is not a blind per-row label

(`ml/pose_cue_annotation_712_v2.json`: row 96 observed as one arm raised overhead, row 72 as arms down). For both rows the previous row's round equals the row's own round: quicknet emits one round every 3 s and adjacent frames of this take are closer than that, so the two published rounds coincide and the lower bound of Proposition 2 is the same beacon that seeded the next transition. Rows straddling a round boundary publish two different values; the relation does not require them to differ. As a regression fixture for the defect that forced the final revision, row 95 of the same session, whose previous-row round is 31,521,619 and whose own round is 31,521,620, was executed under the final ELF against an independent oracle (session tree from the chain log, digests from the audited modules, both networks re-run in Python): 1,101 of 1,101 statement bytes matched with none circuit-derived, 4,149,667,741 instructions, and both rounds decode as distinct values (`final_relation/boundary_row_095/`). That boundary fixture is an execution record, separate from the row-95 fleet proof reported in Section 7.6.

Both rows lie inside the anchored 260-row prefix and share the anchor, receipt, opening and trapdoor. The exact split membership of every row is in the bundle (`ml_splits.json`).

7.6 Every row of the session

The relation is row-generic. We ran circuit-independent oracles for all 712 rows and the final ELF for the 259 rows inside the published prefix. For every row the two frozen networks were re-run in Python from their blobs and the typed digests recomputed by the audited modules (the same circuit-independent oracles as for the proved rows), and every row inside the anchored prefix (rows 1 to 259; row 0 has no predecessor, and rows 260 to 711 lie outside the 260-row prefix the August anchor covers) was executed under the final ELF against its own oracle. All 259 executions matched their oracle on all 1,101 statement bytes with none circuit-derived. Instruction counts ran from 4,146,753,922 to 4,154,781,545 (median 4,150,763,529) and grow by about 31,000 per unit of row index, because the anchor legs scan the chain-log prefix linearly to find the row and its predecessor. The 257 anchored rows not proved in ceremony 2 were then Groth16-proved under the same key on an eight-instance fleet (Section 7.1); all 257 proofs verify with the standalone verifier, each proved statement is byte-identical to the executed statement above, and five (rows 1, 95, 130, 200 and 259) were also verified cold with the SDK against the reproducible ELF and their own oracles, so every one of the 259 anchored rows carries a proof under vkey `0x0019d16c...0490` whose statement matches its circuit-independent oracle on all 1,101 bytes (`all_rows/proofs/`, `PINS_all_rows.json`).

Across the 712 rows the matched coupling numerator ranged from $-1,924,590$ to $93,030,486$ (median $49,650,008.5$); two rows, both outside the anchored prefix, score negative, and the lowest anchored row is row 95 at $2,910,142$. The 72 architecture-selection rows have a median of 49.63 million against 49.65 million for the other 640, as they should, since these are matched scores. The pose verdict agrees with the cue annotation on 532 of the 577 hold-confirmed rows (the eleven classes are cues 01 to 11, with the `12_freeze` cue, hold the last pose, carrying class 10 in the training labels): 431 of 461 training rows and 101 of 116 evaluation rows, the latter being the agreement with the cue-aware annotation reported in Section 8.2, with the weakest cues `09_hands_up` (38 of 50) and `02_superman` (46 of 55). The session spans 66 distinct quicknet rounds ($31,521,605$ to $31,521,705$); 65 rows straddle a round boundary, 25 of them inside the anchored prefix, so the two published rounds differ on 25 of the 259 anchored rows and coincide on the rest. Figure `all_rows_session` plots all four quantities per row. The oracle values and per-row membership records for all 712 rows, the executed statements for rows 1 to 259, and the figure are in the bundle (`all_rows/`). These per-row numbers describe the fixed functions on this take; the caveats of Section 8 apply to every one of them.

8. The learned components, as diagnostics

The proof binds a frozen function's output to its input; it certifies the identity and exact evaluation of each pinned function, but nothing about its accuracy or physical meaning. This section reports what has been measured, with the sampling unit and the confounds. The pose diagnostics use one subject, one rig and one take; the coupling diagnostics additionally include the verification take reported in Section 8.1; the realness measurements of Section 8.1 additionally use held-out sessions `d2` and `v10` and a forger trained on April data.

8.1 Coupling discriminator (trained-r32 PTQ v2)

Model. Five 3×3 integer convolutions with strides 1, 2, 2, 2 and 2 (24, 32, 48, 64, 64 channels) and a 1×1 head, input the 8×8 -reduced camera ($4 \times 32 \times 32$) and emission ($3 \times 32 \times 32$) concatenated; int16 activations, per-layer requantisation, int64 accumulation; score = sum over the 2×2 head field, numerator over 4. *Training* (float, then post-training quantisation): AdamW, learning rate $1e-4$, weight decay 0.01, betas (0.9, 0.999), eps $1e-8$, batch 8, 200 epochs, gradient clip 1.0, seed 20260823, objective matched-vs-crossed logistic plus a four-arm coupling softplus term; negatives by a far derangement of emissions with minimum donor distance 12 rows; 424 training rows; float32 on one GPU. *Splits.* Development rows only: 424 fit rows and 72 architecture-selection rows (96–119, 320–343, 544–567); there is **no untouched test set on this**

session. The ladder record for this exact checkpoint reports `selection_performed: false`, `resolution_winner: null` and `status candidate_not_frozen`: the 72 rows were reserved for architecture selection, but no selection among candidates was recorded on them; r32 is the candidate the principal chose to freeze, not a recorded winner. The 72 selection rows are the only rows with both matched and crossed ladder numerators in the 144-value fixture. On them the matched numerator exceeds the crossed numerator of the same row on 72 of 72 rows (paired), while the pooled area under the ROC over all 72×72 matched/crossed pairs is 0.9983 (9 of 5,184 pairs inverted): one matched numerator is negative ($-1,892,996$) and two crossed numerators are positive (largest 20,290,152), so the separation is not perfect. The float checkpoint's training record gives the same pooled AUROC (0.99826) with one matched example in the 0.3–0.4 probability bin and two crossed above 0.5; the quantised artifact's 144 integer numerators are reproduced bit for bit by the native Rust runner and the Python re-run of Section 6 (`ml/r32_native_parity_results.json`, `ml/r32_parity_144_numerators.json`); among the 72 selection rows, the 24 anchored rows 96 to 119 were executed and proved by the final guest, and every guest numerator equals the fixture's matched numerator; separate circuit-independent oracle records give coupling numerators for all 712 rows (Section 7.6); the float checkpoint has no integer numerators and is compared only through its recorded statistics. An earlier draft stated AUROC 1.0; that was wrong. A separate 288-row verification session was scored once, on 6 September 2026, under the preregistered protocol reported in the next paragraph. The measured property is **conditioning** discrimination (right pattern against wrong pattern on real frames), not realness.

One look at the verification session. The 288-row verification session, recorded on 22 August 2026 about 225 seconds after this session ended on the same apparatus, subject and room, was scored exactly once, on 6 September 2026, by the frozen trained-r32 PTQ-v2 integer model under a preregistered single-opening protocol: the evaluation program, its donor maps and its acceptance criteria were frozen first (freeze digest `8e95845bdacb0a95973ed2fb106ec8ef711a8402271c55d01bf2b687bd02a03f`, 32 pinned files), the opening was authorised by one sentence submitted at the owner desk and recorded in a journal that the program re-resolved before reading a verification row, and an exclusive claim outside the package made the frozen program refuse a second opening. The frozen criteria were: pooled AUROC at least 0.95; at most 14 of 288 false accepts and 14 of 288 false rejects at the integer numerator threshold 3,727,229; at least 137 of 144 reciprocal donor pairs ordered correctly in both directions; and, for an equal-budget shuffled-emission training control scored in float, pooled AUROC and directional ordering rate each within $[0.4, 0.6]$, separately under each of the two donor maps (adjacent-row and 12-row). The result: AUROC 0.99988 with 1 false accept and 4 false rejects under the far map and 0.99778 with 7 false accepts and 4 false rejects under the near map; 144 of 144 reciprocal pairs correct in

both directions under each; control AUROCs 0.4986 and 0.4960 with ordering rates 0.4931 and 0.4965; coverage 288 of 288 rows and 144 of 144 pairs; no integrity problem, run error or failed criterion; verdict PASS. This establishes the frozen model's conditioning separation on one further take of the same rig, minutes later, and nothing about another apparatus, room or subject, liveness or adversarial resistance; the proof of this paper covers the development session's anchored rows and not the verification session. The frozen package, the run records and the verdict are released separately as the one-look evidence package.

Realness (exploratory). One generator-based adversary class (frames produced by a diffusion forger at its step-100,000 checkpoint from the real frame two rows earlier) gave zero fakes passing and zero reals rejected on six real against six forged test subsequences per evaluation seed, three evaluation seeds sharing the same underlying splits, a bootstrap interval [1.0, 1.0] that is degenerate rather than informative, because the empirical bootstrap resamples five perfectly separated observed clusters per class and so expresses no population uncertainty; one checkpoint run was completed where the predeclared plan asked for three forger seeds; its training seed is unrecorded, and the configured seeds 11, 22 and 33 were never trained. The project note records that a generator-free replay-and-splice diagnostic and a black-box ℓ_2 probe were also run on the rented box; their run records were not preserved when it was terminated, so no figures from them are reported here (`ml/realness/README.md`). The adaptive white-box attack (class 4) is **outstanding** (it exhausted a 40 GB GPU twice); no result is claimed for it. The generated-fakes record is archived (`ml/realness/`), with the harness package and a note on the discrepancy between its run metadata and its configuration; the exact generating script recorded by the run (`fdb8745d...`) is not in the bundle, so the run cannot be reproduced from the supplied harness. These are diagnostics on a small sample; they are not a security argument [realness results v2.0]. Two recorded defects: the predeclared class 2 (autoregressive forgery, each fake conditioned on the previous fake) was never built, so the coverage is one generator class plus one generator-free diagnostic; and the produced ROC artifact carries a self-contradictory sidecar (its label says generated-realness while an inherited explanation field says generator-free); the original record is retained unmodified and a correction companion (`REALNESS_ROC_generated_fa_v1_step100k.CORRECTION.json`) supersedes the sidecar for interpretation. The numbers above come from the run record's scores, not from the sidecar text.

8.2 Pose classifier (uncr64 PTQ)

Model. Five 3×3 stride-2 integer convolutions (8, 12, 16, 24, 24 channels) and an eleven-class 1×1 head on the whole frame reduced to $4 \times 64 \times 64$; int16 activations; verdict = first maximum of the eleven int64 logit sums. *Training* (float, then PTQ):

AdamW, learning rate 1e-3, weight decay 0.01, betas (0.9, 0.999), eps 1e-8, batch 32, 64 epochs, no early stopping, unweighted cross-entropy, gradient clip 1.0, float32 on CPU, identical initial parameters and minibatch order across arms; five initialisations for the interleaved arms, three for the temporal-separation arms. *Split*. Within-class interleaved: 461 training and 116 evaluation rows over 577 annotated rows and eleven classes (`ml_splits.json`). *Result on that split*. The annotator saw each cue, so the labels are cue-aware block annotations, not independent blind pose labels. The integer model agrees with the float model on 114 of 116 evaluation rows and with the cue-aware annotation on 101 (float: 102); across five initialisations the whole-frame arm's median is 0.879 (102 of 116). Per-initialisation agreement counts, whole frame: 102, 93, 104, 102, 92 (crop: 114, 116, 111, 113, 113; performer window: 111, 108, 109, 109, 104; wide window: 110, 102, 104, 105, 106; background only: 53, 73, 60, 73, 17).

Initialisations, in that order: the ladder's common initial state (`common_initial_model_state_sha256` in the pose training record) and `torch.manual_seed` 1, 2, 3, 4 re-initialisations of the convolution layers; the proved checkpoint is the manual-seed-3 whole-frame model (102 of 116 in float; sha256 `b69f7ce9...10fb`), the median-agreement initialisation of the five, selected by a pick rule recorded after the spread outcomes were known (`ml/pose_seed3/PICK.json`); the re-run that saved the five checkpoints reproduced the five counts exactly as its positive control, and its configuration is `ml/pose_seed3/pose_training_config.json`. The temporal-separation arms use the common state and seeds 1 and 2. *Classwise, integer model* (n / agreeing with the annotation): class 0 7/4, 1 11/9, 2 11/9, 3 10/10, 4 10/9, 5 10/10, 6 10/10, 7 10/9, 8 10/6 (float 7), 9 10/9, 10 17/16; four of the fifteen errors fall on class 8 and three on class 0 (the full matrix is `pose_confusion_116.json` in the bundle). *Model selection*. The proved model is the whole-frame arm at the 64×64 reduction because a proof over a hand-placed window depends on the window; the crop arm is the earlier proof's model, and the performer-window and background-only arms are synthetic diagnostics, not candidates [pose result v2.0].

The confound. Each class occupies one contiguous run of rows (class 0: 48–84; ...; class 10: 625–711), so class and time-in-take are perfectly confounded and no split of this take can separate them. The interleaved split puts 112 of 116 evaluation rows one frame from a same-class training row. A synthetic background-only arm (performer blanked) still reaches a median 60 of 116 (range 17 to 73) against a majority-class baseline of 17. The pose result record [pose result v2.0] records further confounds: crop and wide use the published block-mean reducer while whole-frame, performer-window and background-only use a separate anisotropic reducer that warps the rectangular frame to square and gives unequal cells equal weight, so cross-family differences conflate field of view with resampling; the masked performer-window and background-only inputs are out-of-distribution diagnostics; adjacent rows are

temporally dependent; and the initialisation spreads measure optimiser variance, not uncertainty across takes.

Temporal separation (predeclared, then audited). Splitting each class run into halves, training on one half and evaluating on the other, raises the median distance to the nearest same-class training row from 1 to 14 rows but leaves a minimum of 1. Medians of three initialisations, fraction agreeing with the cue-aware annotation (chance 0.091):

arm	early-train (287 train, 290 evaluation; n=3)	late-train (290 train, 287 evaluation; n=3)
crop (published 1,024-square window)	0.986	0.878
performer window (synthetic)	0.797	0.714
whole frame (the proved model's arm)	0.734	0.537
background only (synthetic)	0.290	0.150
shuffled-label control (n=1)	0.117	0.150

The whole-frame arm is the most seed-sensitive (262/213/208 of 290 early; 143/154/187 of 287 late); the background-only arm stays above the shuffled control in the early direction and equals the majority-class baseline in the late direction; the direction asymmetry (8 to 20 points: 10.8 crop, 8.2 performer window, 19.8 whole frame, 14.0 background) is descriptive rather than controlled: the two directions use different evaluation rows, class composition and half-run difficulty. Crop survival does not establish that the network reads a body rather than run-persistent illumination or scenery inside the window. The external project audit records that the pre-analysis note was timestamped about 112 s before the result; neither the note nor its timestamp evidence is in this bundle, and it is not an immutable preregistration [temporal separation v2.0]. **Conclusion.** The pose verdict is the output of a fixed function on a fixed frame and nothing more is claimed for it; separating pose from time requires a capture with repeated cues in shuffled order, across sessions and subjects.

8.3 What the proof adds to a learned output

Whatever the models are worth, the proof establishes that their outputs were computed by exactly the pinned functions on exactly the frame whose hash entered the chain, and that the pattern paired with each frame was derived from a chain state seeded by beacon rounds verified in circuit. A reader may disregard the models'

semantic labels while retaining Theorem 1's execution claim; replacing either model requires a new ELF, verification key and proofs. The physical inference of Section 9.2 depends on their empirical validity; Theorem 1's byte-level execution claim does not.

9. Security analysis

9.1 Assumptions and the binding theorem

Let R be the relation of Section 5.2 over public statement x and private witness $w =$ (row header, membership witness, raw frame, σ_t , anchor transaction bytes, anchor Merkle branch, anchor block header, incoming viewing key, chain-log prefix, PREFIX.json, receipt, opening, trapdoor), the thirteen inputs the guest reads.

Assumptions:

- **A1 (zkVM).** SP1 6.4.0's core STARK, recursion and Groth16 wrapper are knowledge-sound for each pinned ELF invoked below (the row relation's and the chain relation's): an accepting proof implies a witness satisfying the program's checks, up to the soundness error of the Fiat-Shamir transform and the Groth16 CRS assumption.
- **A2 (build).** Each pinned verification key is that of its corresponding audited sources compiled with the pinned toolchain and lockfile (Appendix A), and the verifier obtains it from a trusted channel.
- **A3 (hashes).** BLAKE3, BLAKE2b-256 and SHA-256 are collision resistant.
- **A4 (beacon).** Quicknet signatures are unforgeable (BLS, co-CDH in BLS12-381); the signature on a message under a fixed public key is unique (exactly one group element verifies), which we state as an explicit assumption rather than infer from determinism; and the threshold group releases no round's signature or share before its scheduled time.
- **A5 (commitment).** The extracted Orchard note commitment ExtractP • NoteCommit (Sinsemilla, x-coordinate extraction) is binding, the property the Orchard specification asserts under the discrete-logarithm assumption in Pallas; ChaCha20-Poly1305 decryption under a fixed key is deterministic. The AEAD is not assumed key-committing.
- **A6 (Zcash).** The verifier independently checks that the statement's published header hash equals the hash of canonical Zcash mainnet block 3456294, and accepts only once that block lies beyond the verifier's chosen reorganisation depth.
- **A7 (group).** Discrete logarithms in the prime-order Baby Jubjub subgroup are hard.

- **A8 (early-pattern unavailability; an application premise, not a proven cryptographic assumption).** No party who has fixed the predecessor record $\rho_{t-1} = (S_{t-1}, \text{raw}_{t-1}, \text{meta}, r_{t-1})$ before round r_{t-1} 's scheduled release outputs $E_t = \text{render}(\text{XOF}(\text{advance}(\rho_{t-1}, \text{SHA-256}(\sigma_{t-1}))))$ before that release. We believe it because σ_{t-1} is unavailable before release (A4, with uniqueness) and because, treating SHA-256 and the BLAKE3 advance and XOF as random oracles, E_t is a deterministic function of an unqueried oracle output. We have not turned that belief into a cryptographic assumption: the renderer may be many-to-one and we have proved no min-entropy bound for its output; we have written no sampled security experiment with a security parameter, query bounds and an ordering of commitment, oracle access, early output and release; and for the one fixed deployed instance a program that hard-codes E_t violates any universally quantified version trivially. A8 is therefore the premise under which Proposition 2 is read, and Proposition 2 is not claimed as a contribution beyond making that dependence explicit.

Theorem 1 (execution binding). Under A1–A3, if the verifier accepts (π, x) under the pinned key, then, except with the zkVM soundness error and the hash-collision advantage, there exist witness values such that every equality in Section 5.2 holds; in particular the published coupling numerator, pose verdict and logit sums are the frozen networks' outputs on the frame whose BLAKE3 is bound into S_{t+1} and into leaf_t ; the published rounds are those whose signatures verified; and the published txid and header hash are the ZIP-244 identifier of the transaction bytes and the SHA-256d of the header reached by the branch. *Proof sketch.* A1 gives the witness; each equality is a check the guest performs on values it computed; A3 makes alternate digest openings infeasible for a polynomial-time prover; A2 ties the key to the checks.

9.2 The beacon lower bound (conditional)

Proposition 2 (beacon lower bound on the pattern; conditional on premise A8).

Under A1–A4 and A8: an accepting proof with published round r_{t-1} implies that E_t , the pattern the proved frame was checked against, is exactly the value A8 says was unavailable before round r_{t-1} 's scheduled release; so any party who produced E_t before that time either held the round's signature early (contradicting A4, whose uniqueness clause makes any verifying value the signature) or violated A8. *Proof.* Immediate: by A1 the accepting proof has a witness in which σ_{t-1} verifies for round r_{t-1} , $v_{t-1} = \text{SHA-256}(\sigma_{t-1})$, $S_t = \text{advance}(\rho_{t-1}, v_{t-1})$ and $E_t = \text{render}(\text{XOF}(S_t))$ (Section 5.2), which is the value named in A8. The signature is computationally unavailable before release, not non-existent: the signing key determines it at all times. *Scope.* A8 fixes the predecessor record before release. A party who chooses ρ_{t-1} after seeing σ_{t-1} is outside A8. The session-chain

proof of Section 9.5 verifies the earlier transitions but does not prove when ρ_{t-1} was fixed, so it does not close that case.

Physical corollary (conditional). Fix a threshold τ before applying the corollary. If, additionally, (P1) the proved raw_t is genuine sensor output of a physical scene, not injected or synthesised bytes, and (P2) the published numerator q_t satisfies $q_t \geq \tau$ and, for the relevant acquisition distribution, that event implies that the illumination recorded in raw_t was E_t rather than another pattern or none, then the light recorded in raw_t was emitted after round r_{t-1} 's scheduled release, by Proposition 2. Neither P1 nor P2 is established by the proof: P1 fails under raw injection and synthesised frames, and P2 is an empirical property that the diagnostics of Section 8.1 measure but do not discharge. The corollary bounds *when* the recorded light existed and nothing else: a live optical relay of a remote or displayed scene lit by E_t satisfies both premises, so nothing here excludes relay or authenticates where the scene was or what it contained. An alternative route, a trusted rig that obtains and displays E_t only after verifying the released signature, would give the same conclusion by assumption; we do not take it, since not trusting the rig is the point. Without P1 and P2 the proposition bounds the computation, not the capture. The row's own verified round r_t bounds S_{t+1} and therefore the following frame in the same conditional way.

9.3 Memo binding

Proposition 3. Under A3 and A5, for the pinned transaction (fixed by its txid under A3) and action index, any incoming viewing key that makes the memo leg accept yields the same `binding=` value.

The 64-byte incoming viewing key encodes the diversifier key dk and the scalar ivk ; orchard's prepared key uses only the scalar, so two encodings that differ only in dk induce the same decryption and it suffices to consider distinct scalars. *Lemma 3a (accepting keys agree).* Let $ivk \neq ivk'$ (as Pallas scalars) both make the leg accept, decrypting the fixed ciphertext to plaintexts $(d, v, rseed, memo)$ and $(d', v', rseed', memo')$. For each plaintext the leg derives, as orchard 0.15.3 does for a version-3 note: ρ from the action's old nullifier (common to both), $g_d = \text{DiversifyHash}(d)$, $pk_d = [ivk] \cdot g_d$, $\psi = \text{ToBase}(\text{PRF}^{\text{expand_rseed}}(0x09 \parallel \rho))$, $esk = \text{ToScalar}(\text{PRF}^{\text{expand_rseed}}(0x04 \parallel \rho))$ and $rcm = \text{ToScalar}(\text{PRF}^{\text{expand_rseed}}(0x0B \parallel g_d \parallel pk_d \parallel v \parallel \rho \parallel \psi))$; it then requires $\text{ExtractP}(\text{NoteCommit_rcm}(g_d, pk_d, v, \rho, \psi))$ to equal the action's cmx and $[esk] \cdot g_d$ to equal the action's epk . Two accepting scalars therefore give two openings of the same extracted commitment. By binding (A5) the openings coincide, in particular $g_d = g_{d'}$ and $pk_d = pk_{d'}$; then $[ivk] \cdot g_d = [ivk'] \cdot g_d$ with $g_d \neq 0$ in a prime-order group forces $ivk = ivk'$. Hence, except with the

negligible probability with which a polynomial-time prover breaks A5, at most one scalar accepts; the statement is computational, not a mathematical impossibility.

Proof of Proposition 3. By Lemma 3a the accepting scalar is unique; with epk fixed by the transaction, the AEAD key $K = \text{KDF}([\text{ivk}] \cdot \text{epk}, \text{epk})$ is then fixed, and decryption under a fixed key is deterministic (A5), so the 564-byte plaintext, and with it the memo and the `binding=` value parsed from it, is fixed. The argument never assumes the AEAD is key-committing: a prover who minted the transaction could arrange one ciphertext that authenticates under two keys, which is why the binding routes through the note commitment rather than the tag. The leg also requires the esk derived from rseed to regenerate the action's epk; that fixes epk's relation to the plaintext but is not needed for uniqueness. *Note.* The argument is collision-style (two openings of one commitment), not a second-preimage one, because the prover chose the transaction; A5 as stated (binding) is the right assumption for it. The viewing key therefore need not be published for soundness.

9.4 The anchor as a receipt

Proposition 4. Under A1–A3, A5 and A6, an accepting proof implies that canonical block 3456294 contains the proved transaction's action; that its fixed ciphertext has the unique accepting memo binding of Proposition 3, equal to SHA-256 of the proved receipt; and that $C = m \cdot \text{Base8} + r \cdot Y$, where r is the proved opening and m is the Section 5.2 hash-to-scalar of the canonical `{contentRoot, network, profile, recordScope}` tuple, with `contentRoot` computed from the proved `PREFIX.json`. *What this does not imply.* Under A7 alone, C binds its opening only against parties who do not know $\text{td} = \log_{\{\text{Base8}\}} Y$. The prover holds td and proves it (flag 1), so **for the prover, C binds nothing**: an alternate opening to any other prefix can be produced after the block. The anchor is therefore an inclusion receipt for the receipt digest with a disclosed contingency, not an upper bound on when the prefix existed. Nor does it become one for a verifier who lacks td : what matters is whether the party able to supply the record held td , and the operator did, so any later verifier may be shown an opening minted after the block and cannot tell it from the supplied opening. The sound statements are that canonical block 3456294 contains the proved transaction ciphertext; that, under A5, its unique accepting memo binding equals SHA-256 of the proved receipt; and that the demonstrated chameleon opening exists. Publishing td later makes the equivocation capability public without changing this. We keep the anchor because it is real, cheap, and correctly labelled; a future protocol version mints a plain hash commitment alongside C .

9.5 Row-to-session binding, and the session chain proved

The leaf binds S_t , S_{t+1} , both digests, meta, r_t and v_t ; the path binds position t under the committed root (A3). The per-row relation verifies the transitions $t-1 \rightarrow t$ and $t \rightarrow t+1$. A second program proves the chain as a whole: it reads the complete chain log (367,429 bytes, 712 rows), verifies one quicknet signature for each of the 66 distinct rounds (the signature logged on the first row of that round) and checks that every row's logged beacon value equals the verified value of its round, which is SHA-256 of that signature (the signature field of a later row in the same round is not itself re-verified; in this log every row of a round carries the same signature), recomputes every one of the 712 transitions $S_t \rightarrow S_{t+1}$ with `advance_chain` from the logged raw digest, metadata, round and value (the last landing on S_N), requires the rounds to be non-decreasing, and rebuilds the ordered-session tree exactly as Section 5.2 defines it (context, leaves, padding leaves, nodes, wrapped root). Its 279-byte statement publishes the row count, depth, number of distinct rounds, first and last rounds, S_0 , S_N , the chain-log BLAKE3, the manifest digest, the context digest, the root and the session identifier; the context digest and root equal the ones in every per-row statement, so the tree the per-row proofs open into is the tree of this verified log. The program executes in 368,844,955 instructions (324 million of them the 66 BLS verifications) and was Groth16-proved in 477.7 s on an A100 (setup 20.0 s), with the guest ELF 4934b9e2...3926 and key 0x005402a8...df50 reproduced byte for byte on the proving machine, cold-verified with the SDK on the development machine and standalone-verified with `sp1-verifier` (transcripts `final_relation/chain/logs/halo_sdk_cold_verify_20260903.txt` and `halo_standalone_verify_20260903.txt`; the guest ELF is bundled as `final_relation/chain/zeebeam_chain_guest.elf` and the crate as `source/rust/zeebeam_chain_sp1_candidate/`); all 279 statement bytes match an oracle built without the circuit (`final_relation/chain/chain_expect.py`, which runs from inside the bundle). **What this proves and does not.** The log is internally consistent and beacon-seeded at every step, and the session tree is its tree. The operator supplied the log: nothing here says that it is an authentic acquisition chronology, that its raw-frame digests name frames a camera produced (Section 9.2), or that its metadata and round schedule were committed anywhere before the session ran; the anchor of Section 9.4 is the only external commitment, and it is a receipt. Re-hashing the frames themselves would cost about 500 billion instructions (712×702.7 million); the per-row proofs do that for the rows they cover.

9.6 Key pinning and verifier distribution

A verifier must obtain the vkey hash from a trusted channel and may re-derive it from the pinned ELF with the SP1 SDK; Appendix A pins the ELF, lockfiles and build recipe;

Section 7.3 records that build paths change the key, and the final build removes them.

9.7 What the statement reveals

The statement contains no pixels. Apart from the fixed headers, denominator, flags and the model, provenance and beacon constants listed in Section 5.1, its instance-dependent fields reveal: the row index, row count, tree depth, session identifier, S_0 , S_N , the authority-manifest and chain-log digests, the context digest and session root, both beacon rounds (scheduled release times; under Proposition 2's premise a lower bound on emission, never a capture-time estimate or an upper bound), the coupling numerator, the pose verdict, the head-saturation count and the eleven logit sums, the typed context and root, the uncropped commitment, the anchor txid and block hash, contentRoot, the receipt digest, C and the trapdoor flag. It does not reveal the row's chain digests or the frame.

10. Related work

The systems closest to ours differ in what they trust and in which of three attacks they address: *replay* (an old recording presented as new), *relay* (a live recording of a screen, print or remote scene) and *substitution* (synthetic bytes injected as sensor output). Hardware-rooted capture and C2PA [C2PA] sign at the sensor or the origin: they exclude digital substitution only as far as the device key and firmware are trusted, do not exclude optical relay (a trusted camera can photograph a screen), and bound time only by a device clock or an external timestamp. Zero-knowledge image provenance [Naveh and Tromer 2016; Kang et al. 2022b; Datta et al. 2024] proves that a published image is a permitted transformation of a signed original, so it inherits exactly the original's guarantee and adds none against the three attacks.

Timestamping [Haber and Stornetta 1991; OpenTimestamps] gives an upper bound on when a digest existed and nothing about capture. Active-illumination liveness [Gerstner and Farid 2022] and learned anti-spoofing [Yu et al. 2021] address relay with detectors whose outputs must be trusted by whoever runs them, and exclude digital substitution only if the detector's input path is trusted. Our relation's execution binding trusts no sensor and no detector operator; its physical corollary trusts the sensor exactly to the extent of $P1$, and says so. Under $A8$, $P1$ and a thresholded $P2$ it rules out only a frame acquired before r_{t-1} 's scheduled release; it does not exclude replay of a frame acquired after that release, because it proves no upper bound, and without $P1$ it does not exclude byte substitution; it does not exclude optical relay of a live scene lit by the current pattern and does not authenticate scene origin, and says so; and it makes the detector's output a proved function of the recorded bytes, so a reader may audit the detector, or replace it at the cost of a new key and

new proofs, without touching the execution claim. Zero-knowledge inference [Liu, Xie and Zhang 2021; Kang et al. 2022a; EZKL] proves a fixed model's output on a committed input; our networks are small instances run as plain code in a zkVM and bound to a capture chain. Randomness beacons [Syta et al. 2017; drand] supply the public unpredictability the lower bound rests on, and we verify their signatures in circuit rather than trusting a relay of the beacon. Chameleon hashes [Krawczyk and Rabin 2000] make our anchor repudiable by design; Section 9.4 states the resulting receipt with its contingency.

11. Limitations and future work

- The physical claim is conditional on acquisition assumptions the proof cannot check (Section 3.3); a trusted sensor path or an empirical relay/injection study is needed to discharge them.
- The pose verdict is confounded by time on the only take; the temporal-separation test narrowed but did not remove this. A multi-session, multi-subject capture with shuffled cues is required.
- The coupling model has no untouched test set on this session; the separate 288-row verification session was scored once under the preregistered protocol of Section 8.1 and passed, which establishes separation on that take of the same rig and nothing wider. The realness measurements are small-sample and one adaptive class is outstanding.
- The anchor gives no record upper bound to anyone, because its custodian held the trapdoor; mint a plain hash commitment next time.
- The renderer is 52% of the instruction count. Rows 260 to 711 lie outside the published 260-row prefix; covering them under the full relation without exercising the disclosed chameleon-equivocation capability needs a new anchor transaction.

12. Reproducibility and artifact

The public release bundle (`bundle/proofs_20260902/` in the release tree) contains: raw proofs and statements, framed proofs, manifests, decoded statements, `PINS.json` (the ceremony-1 pins), `ml_splits.json` (every row's role in both models' data), the proved ELF, the standalone verifier and decoder sources, the recorded outputs and host-specific scripts of the Python re-runs of both frozen networks (the raw frames, model manifests and `zeebeam_science` dependency needed to rerun them are not in the bundle), the 144-value parity fixture and its decoded numerators (`ml/architecture_r32_parity_v2.bin`, `ml/r32_parity_144_numerators.json`), the cue annotation, the pose robustness, temporal-separation and whole-frame ladder records

and both ladder training records (ml/), the frozen source tree with the vendored crate, the two embedded model blobs, the canonical preprocessing configuration the build embeds (source/src/zeebeam_science/preprocess_v1.candidate.json, 764 bytes, sha256 6345dc41...bd6d, digest line source/SRC_INPUT_SHA256SUMS) and the staging and build scripts (source/, with its digest list final_relation/SOURCE_TREE_SHA256SUMS, 38 files, sha256 e45be3c0...37a2; a clean staged build from an empty root on 6 September 2026 reproduced both pinned ELF's and keys, logs in source/clean_build_20260906/), PINS_final.json and the executed 1,101-byte statements of both rows under the final ELF, the pose confusion matrix (pose_confusion_116.json), the coupling and pose training records (ml/coupling_ladder_results.json, ml/pose_ladder_results.json), the 260-row chain-log prefix, PREFIX.json and the canonical receipt (anchor/), VERIFY.md, and SHA256SUMS, which the procedure checks first. Present as well: ENVIRONMENT.md (the locked environment), the ceremony-2 proofs, complete standalone-verifier transcripts for all four proofs from the bundled source built --locked, the boundary-row fixture, the fail-closed build driver source/build_reproducible.sh (set -euo pipefail, cargo build --locked, exactly one ELF, size, SHA-256 and vkey asserted against pins fixed in the script) with its development-machine transcript (the fresh-machine build of the same tree is the ceremony-2 box's, Section 7.3, made with the day's fail-open driver and recorded in its logs), the final-ELF execution logs with every cycle-tracker region (final_relation/execution_logs/), the GPU-memory observation record, and the realness records that exist (ml/realness/: the generated-fakes ROC record, the harness code and configuration; the replay/splice and l2 run records were not preserved and Section 8.1 says so), the 257 fleet proofs of the remaining anchored rows with their manifests, raw proofs, statements, per-box prover logs, the standalone-verification record and PINS_all_rows.json (all_rows/proofs/, Section 7.1), the oracle values and per-row membership records of all 712 rows and the executed statements of rows 1 to 259 (all_rows/), and the chain relation's proof, records, portable oracle, guest ELF and frozen crate (final_relation/chain/, source/rust/zeebeam_chain_sp1_candidate/, Section 9.5). The source tree carries every path dependency of the proved crates, under source/deps/ and source/rust/preprocess_v1_candidate/ (the zeebeam-b3xof-relation, preprocess-v1, trained-r32 model and relation, and uncr64 model and relation crates), digested with the vendored jubjub in final_relation/DEPENDENCY_SHA256SUMS. The frozen manifests name those crates by the development machine's absolute layout, so they do not resolve in place; source/stage_for_rebuild.sh recreates that layout deterministically from the bundle, source/README.md describes it, and source/build_reproducible_chain.sh is the fail-closed driver for the chain guest. An air-gapped rebuild from the bundle alone has not been rehearsed. The release includes LICENSE; no persistent identifier is assigned in this tree. Verification needs the bundled standalone verifier built --locked; its direct dependencies are sp1-verifier 6.4.0 and sha2 0.10.

13. Conclusion

One relation over one row's bytes, with two beacon signatures, two frozen networks, a session tree and a Zcash receipt inside it, and a statement whose fixed bytes are enforced by the pinned guest and recorded in `PINS_final.json`; proved for two concrete session rows in both its revisions, the final one under a key reproduced byte for byte on the proving machine, and then for every one of the 259 rows the August anchor covers; a second proof covers the session chain those rows belong to. The proof establishes execution binding exactly; the physical reading is conditional and labelled; the models are diagnostics and labelled; the anchor is a receipt and labelled. The claim wording was written under adversarial review, and the relation was changed twice because of it.

Acknowledgements

Adversarial review by OpenAI GPT-5.6 (Codex; high reasoning effort for rounds 1 to 10, ultra from the eleventh pass on), operated by the project as "Sol"; its findings are recorded in Section 7.4. BOSUN drafted this manuscript and ran the ceremony under the principal's direction.

References

- Aumasson, J.-P., Neves, S., O'Connor, J., Wilcox-O'Hearn, Z. BLAKE3: one function, fast everywhere. Specification, 2020. <https://github.com/BLAKE3-team/BLAKE3-specs>
- Boneh, D., Lynn, B., Shacham, H. Short signatures from the Weil pairing. ASIACRYPT 2001, LNCS 2248, pp. 514–532.
- C2PA. Content Credentials: C2PA Technical Specification. <https://spec.c2pa.org/> (accessed 2 September 2026)
- Datta, T., Chen, B., Boneh, D. VerITAS: Verifying image transformations at scale. IEEE Symposium on Security and Privacy 2025, pp. 4606–4623, DOI 10.1109/SP61157.2025.00097 (IACR ePrint 2024/1066).
- drand / League of Entropy. quicknet: chain hash 52db9ba7...e971, scheme bls-unchained-g1-rfc9380, period 3 s, genesis 1692803367; protocol specification and <https://api.drand.sh//info> (checked live 2 September 2026)
- Faz-Hernández, A., Scott, S., Sullivan, N., Wahby, R. S., Wood, C. A. Hashing to elliptic curves. RFC 9380, 2023.
- Gerstner, C. R., Farid, H. Detecting real-time deep-fake videos using active illumination. CVPR Workshops 2022, pp. 53–60, DOI 10.1109/CVPRW56347.2022.00015.

- Groth, J. On the size of pairing-based non-interactive arguments. EUROCRYPT 2016, LNCS 9666, pp. 305–326.
- Haber, S., Stornetta, W. S. How to time-stamp a digital document. Journal of Cryptology 3(2):99–111, 1991.
- Kang, D., Hashimoto, T., Stoica, I., Sun, Y. (2022a) Scaling up trustless DNN inference with zero-knowledge proofs. arXiv:2210.08674.
- Kang, D., Hashimoto, T., Stoica, I., Sun, Y. (2022b) ZK-IMG: Attested images via zero-knowledge proofs to fight disinformation. arXiv:2211.04775.
- Krawczyk, H., Rabin, T. Chameleon signatures. NDSS 2000.
- Liu, T., Xie, X., Zhang, Y. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. ACM CCS 2021 (IACR ePrint 2021/673).
- Naveh, A., Tromer, E. PhotoProof: Cryptographic image authentication for any set of permissible transformations. IEEE Symposium on Security and Privacy 2016, pp. 255–271, DOI 10.1109/SP.2016.23.
- OpenTimestamps. Protocol and client. <https://opentimestamps.org/> (accessed 2 September 2026)
- Succinct Labs. SP1 zkVM v6.4.0, software release of 12 August 2026 (cargo prove build f66b4bf). <https://github.com/succinctlabs/sp1/releases/tag/v6.4.0>
- Syta, E., Jovanovic, P., Kogias, E. K., Gailly, N., Gasser, L., Khoffi, I., Fischer, M. J., Ford, B. Scalable bias-resistant distributed randomness. IEEE S&P 2017.
- Whitehat, B., Bellés, M., Baylina, J. ERC-2494: Baby Jubjub elliptic curve. 2020. <https://eips.ethereum.org/EIPS/eip-2494>
- Yu, Z., Qin, Y., Li, X., Zhao, C., Lei, Z., Zhao, G. Deep learning for face anti-spoofing: a survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022 (arXiv:2106.14948).
- Zcash: Hopwood, D., Bowie, S., Hornby, T., Wilcox, N. Zcash protocol specification, current revision at <https://zips.z.cash/protocol/protocol.pdf> (note encryption §4.20; accessed 2 September 2026). ZIP 224 (Orchard). ZIP 229 (Version 6 transaction format; NU6.3). ZIP 244 (Transaction identifier non-malleability).
- Project records: see Appendix F.

Appendix A. Publication pins

PINS.json in the bundle pins the two proofs made (ceremony 1); PINS_final.json pins the final relation and its ceremony-2 proofs, and the bundle carries both the executed and the proved 1,101-byte statements of both rows, decoded, which are identical. Headline values for the two proofs made: SP1 vkey 0x0092cba2c77fe7fc426bb539add5d0365e4edea340c2f3c040da9bd73cf9a82c; guest ELF sha256 5b3ebfcd...4ce5 (1,160,904 B). PINS_all_rows.json pins the 257 fleet proofs: one vkey, one ELF, per-proof digests and timings. Chain relation (proved; ELF bundled as

final_relation/chain/zeebeam_chain_guest.elf): ELF sha256 4934b9e2...3926 (460,416 B), vkey 0x005402a8...df50, statement 279 B sha256 a6d1ae85...64f8, raw proof sha256 68cab3a0...775e. Final relation (ceremony 2, proved): ELF sha256 8bcadf53...11db (1,171,416 B), vkey 0x0019d16c...0490; raw proofs sha256 b441ede1...fda9 (row 96) and b52dd2ab...888d (row 72); statements sha256 777bbf1f...b26a and 9b560f90...5190. Session ZEEBEAM_MAINNET_BLOCKING_TRAINING_300S_20260822_001, 712 rows, depth 10, ordered-session root 38a484b8...6572, chain-log BLAKE3 754e5716...7f8b, authority manifest 740d752d...d783; quicknet chain hash 52db9ba7...e971 and public key as in PINS.json; anchor txid 8d1672155e98000498070bd76479e0363b8b9ee0b61f0d32a802d35ef5a4f206, height 3456294, block hash 000000000052f2b0c7e7929e16086c3d15f09922b050109ec722c8576ffddf0b, action 0; receipt SHA-256 fae21624...b30b (853 B); PREFIX.json SHA-256 d67ac2ae...f3f4 (603 B); prefix BLAKE3 182a1bf7...458b (134,188 B, 260 rows); contentRoot 7ce89c0f...4c0d; Baby Jubjub a = 168700, d = 168696, L = 2736030358979909402780800718157159386076813972158567259200215660948447373041, canonical Base8; Y and C decimals; frozen r32 blob 0188b5c0...353a; uncr64 blob c95b0072...7ed7; PREPROCESS_V1 spec 6345dc41...bd6d; uncropped spec a3985f69...8e08; bls12_381 fork tag patch-0.8.0-sp1-6.2.0; vendored jubjub diff digest; program and script Cargo.lock digests; rows 96 and 72 rounds 31521620 and 31521616, previous rounds 31521620 and 31521616.

Appendix B. Statement decoding

verifier/decode_statement.py decodes 1,085-, 1,093- and 1,101-byte statements into the named fields of Section 5.1; it is portable (no absolute paths), prints JSON to stdout, writes nothing unless --out DIR is given, and exits 0 on success. The bundled *_statement.json files were produced by it and are reproduced byte for byte by --out.

Appendix C. Verification procedure

VERIFY.md (v2.9): run sha256sum -c SHA256SUMS first; build the standalone verifier --locked; run it on each raw proof with the vkey hash; decode the statement; perform the two out-of-band checks (block hash at height 3456294 on canonical mainnet; rounds r_{t-1} and r_t against the quicknet schedule). The guide states which frame each round bounds: the ceremony-1 statements (ZBAUGST2) publish no round at all, the row's own round r_t being verified privately inside the relation and supplied to the verifier by the pins and records; and that round bounds the following frame's pattern. The final revision publishes r_{t-1} and r_t , and r_{t-1} bounds the proved frame's own pattern. Sections 1c and 1d of the guide verify the chain proof under its own key and loop the standalone verifier over the 257 fleet proofs, comparing each statement with its executed counterpart.

Appendix D. Negative-control history

Section 7.4 lists the final controls. Development history: the direction-byte control that first passed because the host sanitised input; the same-length raw/packed confusion caught by the oracle; the truncated oracle constant; the Base8 forgery found by audit; the off-by-one beacon indexing found by referee review. All are recorded in the state document with dates.

Appendix E. Sources of the relation

`row_binding_join_spl_candidate/join` (beacon verification, chain, emission, preprocess, typed root, coupling, pose), `row_binding_join_membership_spl_candidate/membership` (membership; `zcash.rs`, `memo.rs`, `august.rs` including the previous-row leg), `program` (guest), `script` (host: witnesses, execute, ceremony, export, vkey), pinned by lockfile digests.

Appendix F. Project records cited

Records named below without a `referee_trail/`, `bundle/` or `all_rows/` path are project records that are not included in this release. The authority-manifest preimage is not bundled; only its digest is.

`docs/research/zeebeam_one_proof_architecture_20260901.md` (state of record);
`zeebeam_pose_nocrop_result_20260830.md` v2.0;
`zeebeam_pose_temporal_separation_20260901.md` v2.0; `zeebeam_realness_results_20260901.md` v2.0; `zeebeam_realness_claim_framing_20260828.md`; `zkbeam_profile_r_repudiable_custody.md`;
the two source audits (not bundled); `referee_trail/FINDINGS_round1.md`; the session manifest and chain log; the uncr64 and trained-r32 manifests, parity vectors and ladder results
(`zeebeam_camera_pose_ladder_20260823/real_august_seed20260823_e64_v1/results.json`, which also carries the cue annotations;
`zeebeam_resolution_ladder_20260823/real_august_far12_seed20260823_e200_v1/results.json`);
pose per-initialisation counts (`pose_nocrop_20260830/out/robustness.json`, `temporal_separation.json`); `referee_trail/FINDINGS_round2.md` to `referee_trail/FINDINGS_round11.md`, `referee_trail/FINDINGS_round12_verification.md`, `referee_trail/FINDINGS_round13_closing.md` to `referee_trail/FINDINGS_round19_closing.md`, `referee_trail/FINDINGS_companions_ultra.md` and `referee_trail/FINDINGS_release_ultra.md`;
the final-ELF execution logs (`joined_build_20260901/final_statements_20260902/`); the all-rows run (`all_rows_20260902/`, `all_rows/` in the bundle); the chain relation (`final_relation/chain/`, `source/rust/zeebeam_chain_spl_candidate/`).

Log

- 3.20 (2026-09-06, BOSUN) — title changed to *ZeeBeam: The Zero-Knowledge Beam* on the principal's direction that titles be anchored in the names themselves; the subtitle and every other line unchanged. (The 3.17 entry records the earlier title.)
- 3.19 (2026-09-06, BOSUN) — closing-pass corrections: the Section 8 introduction distinguishes the pose diagnostics (one take) from the coupling diagnostics, which now include the verification take of Section 8.1; the one-look paragraph states the guarantee exactly (development rows are read in preflight; the exclusive claim makes the frozen program refuse a second opening); no other change.
- 3.18 (2026-09-06, BOSUN) — Section 8.1 records the one look taken on 6 September 2026 at the 288-row verification session (preregistered single opening, verdict PASS; numbers read from the run's verdict), replacing the sentence that called the session sealed and unscored; the Section 11 limitation updated accordingly; no other change.
- 3.17 (2026-09-06, BOSUN) — the title given its paper form, *ZeeBeam: One Relation to Bind Them*; the subtitle and every other line unchanged.
- 3.16 (2026-09-06, BOSUN) — rendering fix, no change to the text or the science: the PDF had clipped the final digit of the Baby Jubjub group order L in Appendix A, a 76-digit word the print layout could not break (the Markdown was complete throughout); the release renderer now wraps long tokens and the build checks every token of forty or more characters against the PDF text. The 3.15 and 3.14 entries' section references are corrected to Section 7.4 and Appendix F. Second render pass: the wrapping rule is scoped to paragraphs and list items so table cells keep their numbers whole, and headings stay with their following content (GPT-6 Astra closing pass 6).
- 3.15 (2026-09-06, BOSUN) — the referee trail is published with this release again on the principal's ruling of 6 September (a trail stays where the release delivers a positive result): Section 7.4 and Appendix F point at `referee_trail/`. No technical content changed.
- 3.14 (2026-09-06, BOSUN) — the referee reports are retained as project records (the trail is held on the principal's instruction; Section 7.4 and Appendix F reworded accordingly); Section 12 names the canonical preprocessing configuration the build embeds, its digest line and the clean staged build of 6 September that reproduced both pinned ELF's and keys. No technical content, proof, statement or pin changed.
- 3.13 (2026-09-05, BOSUN) — Sol's nineteenth (seventh closing) pass found no defect and cleared the tree; its report retained as a project record under the

stopping rule it accepted; nineteen passes stated. No technical content changed.

- 3.12 (2026-09-05, BOSUN) — Sol's eighteenth (sixth closing) pass applied: Appendix F lists every closing report; eighteen passes stated. No technical content changed.
- 3.11 (2026-09-05, BOSUN) — Sol's seventeenth (fifth closing) pass applied: it cleared the tree; seventeen passes stated. No technical content changed.
- 3.10 (2026-09-05, BOSUN) — Sol's sixteenth (fourth closing) pass applied: the front-matter status line, which still read v3.8, brought into agreement with the version; sixteen passes stated. No technical content changed.
- 3.9 (2026-09-05, BOSUN) — Sol's fifteenth (third closing) pass applied: no manuscript finding; fifteen passes stated; the pass repaired the release tree's notices and publication script (RELEASE_NOTES 1.7). No technical content changed.
- 3.8 (2026-09-05, BOSUN) — Sol's fourteenth (second closing) pass applied: VERIFY 2.9 in Appendix C; the review effort stated per phase; fourteen passes. No technical content changed.
- 3.7 (2026-09-05, BOSUN) — Sol's thirteenth (closing) pass applied: the memo binding stated as the parsed binding= value; VERIFY version in Appendix C; thirteen passes; the audit-status clause in 7.4 made neutral. No technical content changed.
- 3.6 (2026-09-05, BOSUN) — Sol's twelfth (verification) pass applied: coupling-model strides corrected from the proved source (1, 2, 2, 2, 2; the earlier decline was wrong); abstract states the supplied transaction and the memo binding exactly; "preregistered" replaced by "predeclared" where the evidence caveat applies; verifier described with its two direct dependencies; Section 7.6 bundle contents exact; Section 12 states that the frozen manifests need the staged layout; twelve passes; status note trimmed. No proof, statement, pin or measurement changed.
- 3.5 (2026-09-05, BOSUN) — Sol's eleventh pass (ultra, on the v3.4 publication bytes) applied: Proposition 4 and A6 restated so that canonical inclusion is the verifier's check (A6) and the receipt binding runs through Proposition 3; P2 given a threshold; replay wording corrected (no upper bound); chain-proof scope in 9.2; A1 and A2 for both ELF's; the witness tuple as the guest reads it; 9.7 disclosure list completed; 8.3 model replacement stated correctly; 7.6 and item 7 distinguish oracles from executions; 7.1 idle time 13.5 instance-hours; 8.1 selection-row and guest evidence restated; 8.2 labels called cue-aware agreement, further recorded confounds added, temporal headings corrected, direction asymmetry and predeclaration provenance restated; Section 12 contents corrected and the source tree's path dependencies (five crates) now bundled; verifier dependencies stated exactly; eleven referee passes; bibliography prompts resolved; publication-

status lines replaced; Appendix F points at the published trail. No proof, statement, pin or measurement changed.

- 3.4 (2026-09-05, BOSUN) — named at the principal's direction: ZeeBeam is the system and the release, Dark Lantern the research programme it came from; title, status note and naming paragraph updated. No technical content changed.
- 3.3 (2026-09-05, BOSUN) — renamed at the principal's direction: the project is Dark Lantern and the proved object a ZeeBeam; the phrase "ZK all the things" now names Table 1 (Section 5.2) only; naming paragraph added to Section 1. No technical content changed.
- 3.2 (2026-09-03, BOSUN) — stale status line ("ninth referee round") corrected; no other change.
- 3.1 (2026-09-03, BOSUN) — Sol's round-10 verification pass applied: chain check stated exactly (one signature per distinct round, values checked per row), operator caveat strengthened, chain ELF, crate location, dependency crate and verification transcripts bundled; 7.6 median 49,650,008.5 and 25 anchored boundary rows (65 across the session); ten referee rounds; conclusion covers all proofs.
- 3.0 (2026-09-03, BOSUN) — every anchored row proved: 257 fleet proofs standalone-verified and matched byte for byte to their executed statements; fleet ceremony, network incident and detached resume disclosed (7.1); 7.6, contribution 7, abstract, Section 12, Appendices A and C; VERIFY.md v2.4, ENVIRONMENT.md v1.2.
- 3.0-draft (2026-09-02, BOSUN) — the session chain proved (new 9.5, contribution 6, abstract, Appendix A); every row of the session through the oracles and every anchored row executed (new 7.6, contribution 7, figure); limitations updated; fleet proof count pending.
- 2.9 (2026-09-02, BOSUN) — harvest of Sol's round-8 report (correctness at minor revision; venue weak reject as a judgement): host execution times attributed to their rows; forger-seed wording corrected; correction companion described consistently; generated-fakes run stated as not reproducible from the bundle; eight completed referee rounds cited, Appendix F extended to rounds 6-8.
- 2.8 (2026-09-02, BOSUN) — harvest of Sol's round-7 report (weak reject on significance; correctness close to minor revision): ceremony-1 row-72 RSS corrected; six completed referee rounds; build driver pins fixed in the script and the transcript labelled a development-machine run; exact replay/splice and ℓ_2 figures removed as unarchived; bootstrap interval described as degenerate; realness harness package and metadata note bundled.
- 2.7 (2026-09-02, BOSUN) — harvest of Sol's round-6 report (weak reject on significance; correctness at minor revision): fail-closed build driver and its transcript; ceremony-1 statements correctly described as publishing no round

(Appendix C, verifier comment); stale oracle note in the frozen host disclosed with correction sidecars; cycle-region logs, GPU-memory observations and realness records packaged; RSS in KiB/MiB; frontmatter round.

- 2.6 (2026-09-02, BOSUN) — harvest of Sol's round-5 report (artifact and provenance): audit count corrected to two source audits and five manuscript rounds in 1 and 7.4; ceremony-2 destructor panics disclosed in 7.1; parity sentence narrowed to native/Python 144/144 and guest agreement on executed rows; boundary-row 95 regression fixture (distinct rounds) reported in 7.5; Section 12, Appendices B, C and F aligned with the bundle (portable decoder, frozen source tree, transcripts, environment, VERIFY.md v2.2).
- 2.5 (2026-09-02, BOSUN) — ceremony 2 folded in: the final relation proved for rows 96 and 72 on a second A100 (14 min 17 s, 12 min 30 s) under the byte-reproduced key; cold and standalone verification; proved statements identical to the executed ones; pending-proof wording removed throughout; 7.1 table, 7.2 sizes, 11, 12, 13 and Appendix A updated.
- 2.4 (2026-09-02, BOSUN) — harvest of Sol's round-4 report: Theorem 2 demoted to Proposition 2, conditional on A8 restated as an application premise with its deficiencies listed, and removed as a claimed contribution; physical corollary rewritten with acquisition (P1) and pattern inference (P2) separated and optical relay explicitly not excluded, in 9.2 and 10; Lemma 3a and its Note phrased computationally; parity wording corrected to quantised-artifact parity across three implementations; the ladder record's unrecorded selection disclosed; referee-round metadata and audit count corrected. (The 2.3 entry's "collision parenthetical removed" referred to the DiversifyHash/PRF^{expand} parenthetical, which was removed; the collision Note that remained is rephrased here.)
- 2.3 (2026-09-02, BOSUN) — harvest of Sol's round-3 report (reject this cycle, resubmit after major revision): Theorem 2 restated on an explicit assumption A8 over the complete beacon-to-pattern map with its random-oracle reading and scope stated, uniqueness made an explicit assumption; Lemma 3a rewritten on Orchard's actual version-3 derivations (rcm over g_d , pk_d , v , ρ , ψ ; extracted commitment; epk check) with A5 on the extracted commitment and the collision parenthetical removed; coupling result corrected from AUROC 1.0 to paired 72/72 with pooled AUROC 0.9983 from the 144-value fixture, now bundled; host time given for both runs; related-work guarantees corrected; cue annotation and seed integers sourced; VERIFY.md v2.0 and the source-tree digest recorded.
- 2.2 (2026-09-02, BOSUN) — harvest of Sol's round-2 report (major revision): abstract, contribution 1 and conclusion now say which revision is proved and which is executed only; Theorem 2 restated with the uniqueness clause in A4 and a random-oracle assumption A8, and the "did not exist" wording replaced; Lemma 3a supplies the two-openings argument behind Proposition 3; the third-party

upper-bound sentence in 9.4 withdrawn; 9.5's chain cost recomputed (1.6 billion with committed digests, 500 billion with re-hashing); 9.7's capture-time gloss corrected; statement table made byte-complete (declared length, padding, reserved bytes); cost table on the final ELF with the untracked remainder and a total; Section 8 scope corrected for the realness sessions; class-2 gap and ROC sidecar defect recorded; per-initialisation counts, classwise results and the selection rationale added; asymmetry range corrected to 8-20 points; sizes presented together; related work rewritten as a comparison of guarantees; pins and artifact list updated.

- 2.1 (2026-09-02, BOSUN) — reference list verified against arXiv, ePrint and DBLP: VerITAS given its S&P 2025 venue and pages, Gerstner & Farid and PhotoProof given DOIs, Yu et al. given the TPAMI venue and confirmed author list; internal verification tags removed from the list.
- 2.0 (2026-09-02, BOSUN) — rewritten after Sol's round-1 referee report: claim narrowed to execution binding with the physical reading conditional; definitions and indices; explicit threat-model exclusions; assumptions A1-A7 and Theorem 1, Proposition 2, Propositions 3-4, Section 9.5 on the chain; both beacon rounds published (statement 1,101 B); corrected memo argument; ML sections rewritten as diagnostics with configurations, splits, seeds, the temporal-separation test, the outstanding class-4 attack; exact costs from the current ELF; corrected citations (NU6.3, §4.20, ERC-2494 authors, RFC 9380 authors); artifact gaps listed.
- 1.2 and earlier (2026-09-02) — see previous versions.